

Oriëntatie op AI

Les 6: data science – lineaire regressie en correlatie

2025-2026 Semester 1

HBO-ICT



Agenda

1. Lineaire regressie
2. Kleinste kwadratenmethode
3. Gradient descent algoritme
4. Correlatie



Lineaire regressie

Lineaire regressie is een **statistische methode** om de relatie tussen *feature*- en *target*variabelen te **schatten**.

Met deze schatting kunnen we (onder andere) toekomstige waarden van variabelen **voorspellen** op basis van **historische** gegevens.



Machine learning

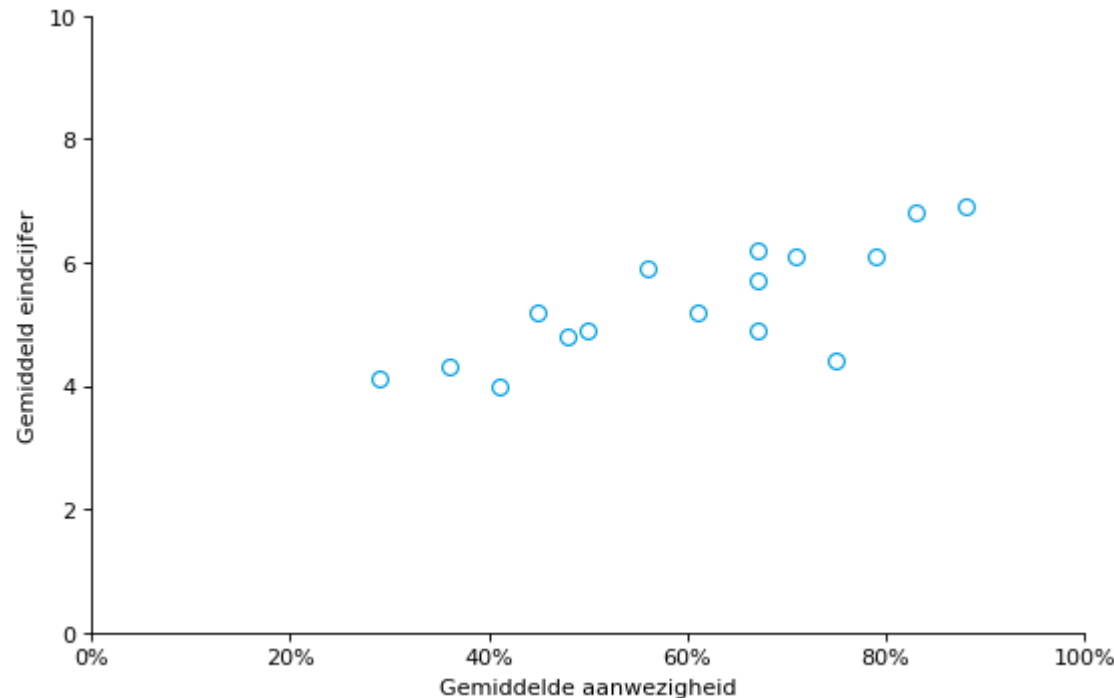
Lineaire regressie is een van de meest eenvoudige en populaire toepassingen van **machine learning**.

Machine learning is een vorm van AI waarbij programma's **leren** van **data**.

Deze programma's worden niet expliciet geprogrammeerd om een taak goed uit te voeren, maar **verbeteren zichzelf** door een **model** aan te passen.

Lineaire regressie

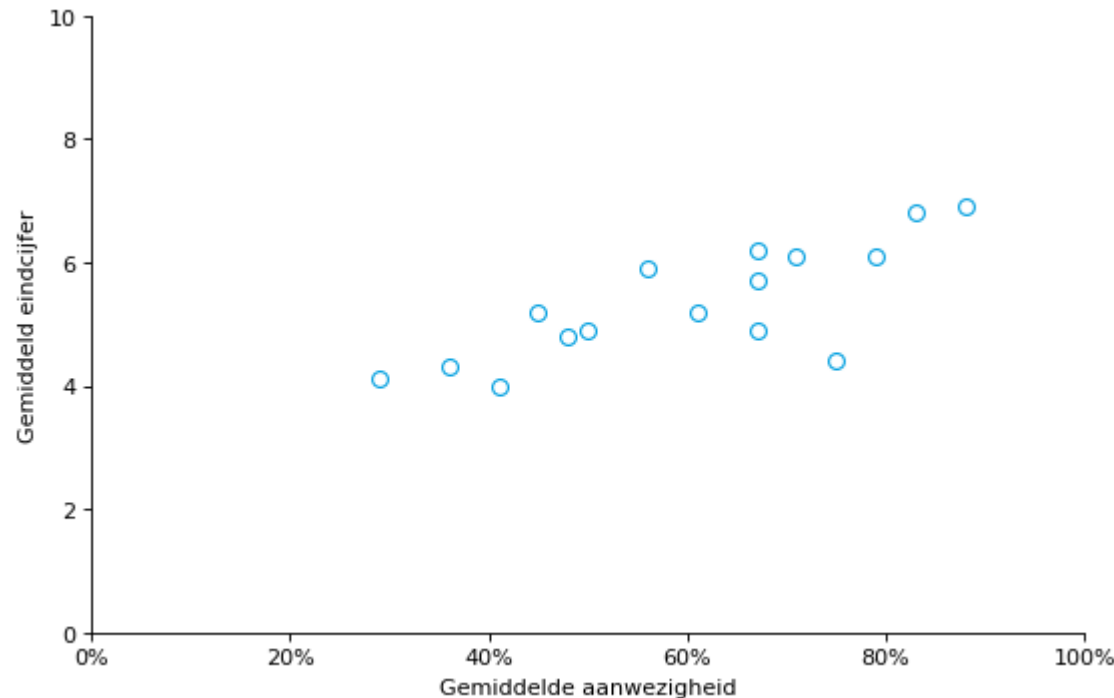
We zouden met lineaire regressie bijvoorbeeld het gemiddelde **eindcijfer** van studenten kunnen proberen te schatten aan de hand van de gemiddelde **aanwezigheid**.



Lineaire regressie

Onderstaande spreidingsdiagram geeft inzicht in de relatie.

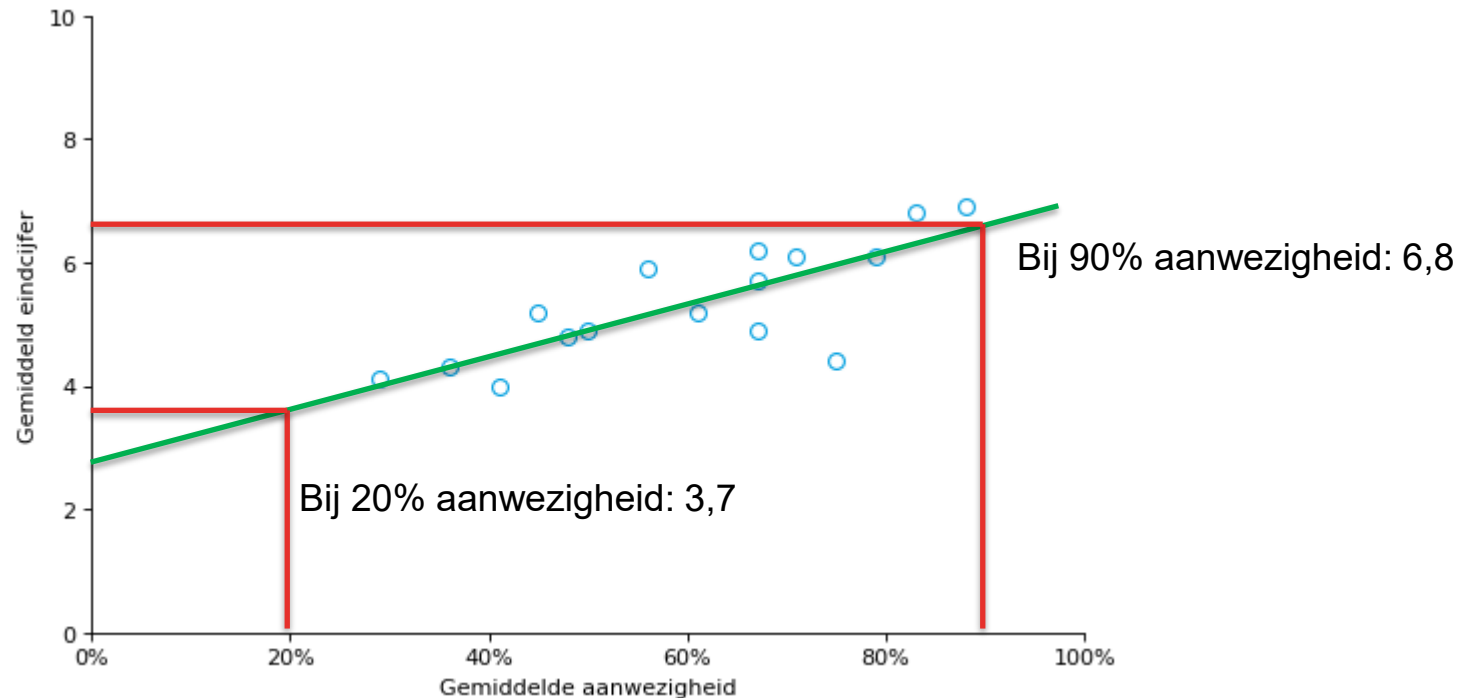
- De gemiddelde aanwezigheid is hier de **feature** variabele.
- Het gemiddelde eindcijfer is de **target** variabele.



Lineaire regressie

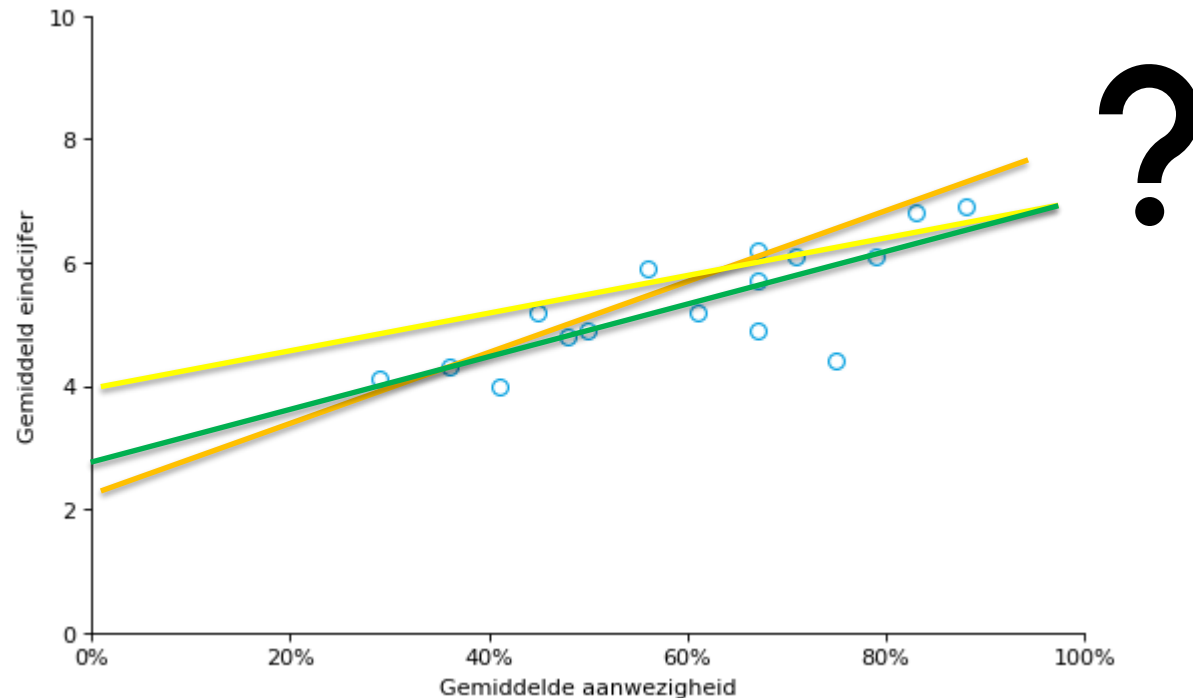
Een intuïtieve manier van schatten: een **rechte lijn** trekken door de puntjes!

Zo kunnen we voor elke aanwezigheid een eindcijfer geven.



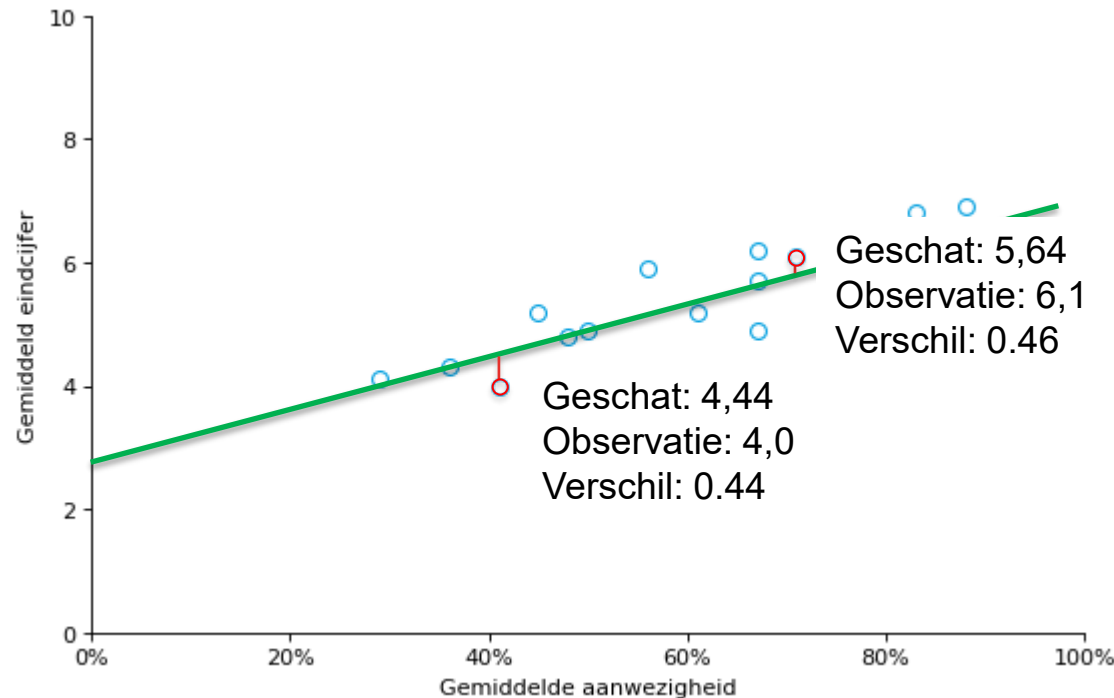
Lineaire regressie

Maar hoe kan je het beste zo'n lijn trekken?



Lineaire regressie

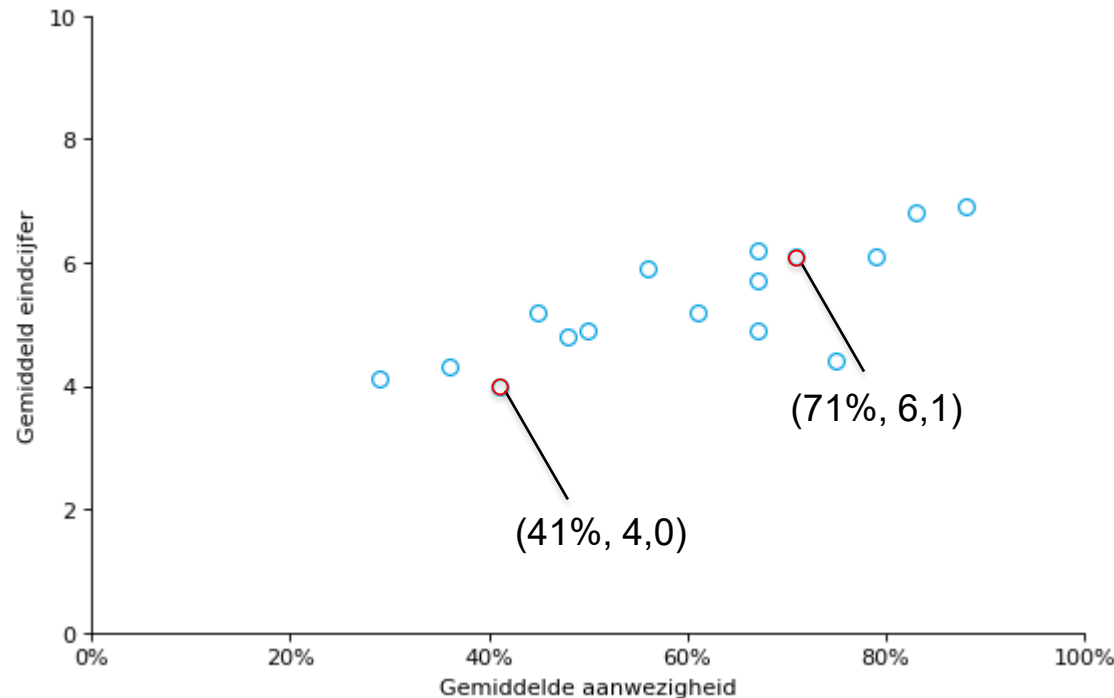
Een mooie benadering krijg je door een lijn te kiezen waarbij de **verschillen** tussen de observaties en de voorspelling **minimaal** zijn.



Kleinste kwadratenmethode

We kunnen hiervoor de **kleinste kwadratenmethode** gebruiken.

- We gebruiken de observaties (x_1, y_1) , (x_2, y_2) , (x_3, y_3) ... (x_n, y_n)

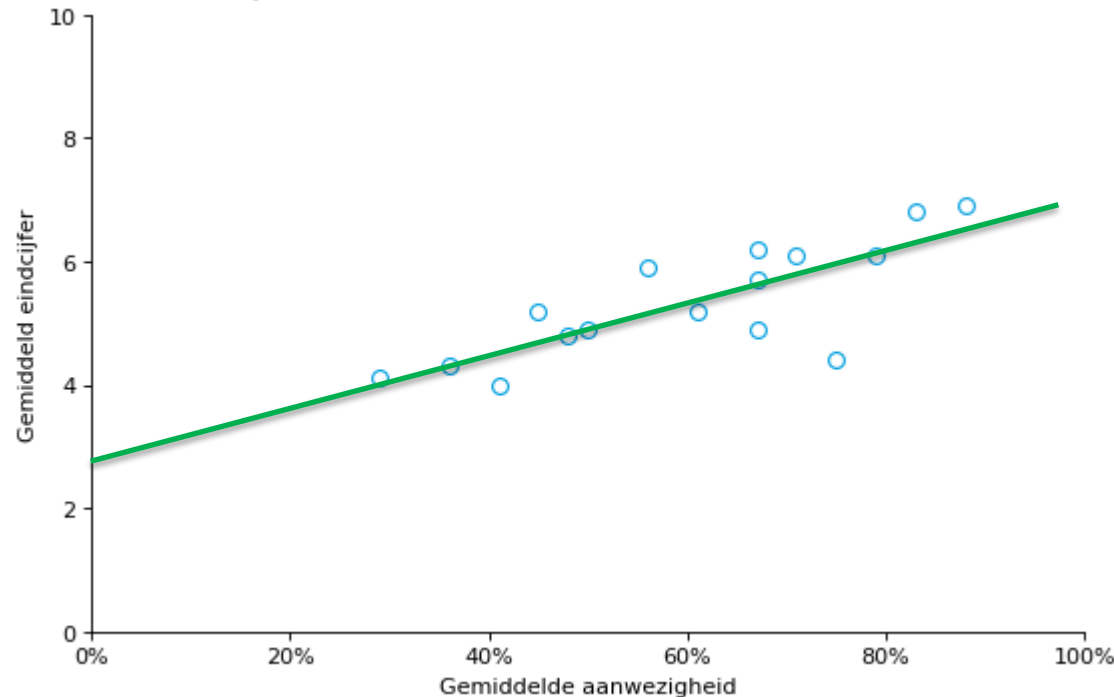


Kleinste kwadratenmethode

We gaan op zoek naar de beste waarden voor de vergelijking

$$\hat{y} = a + b \cdot x$$

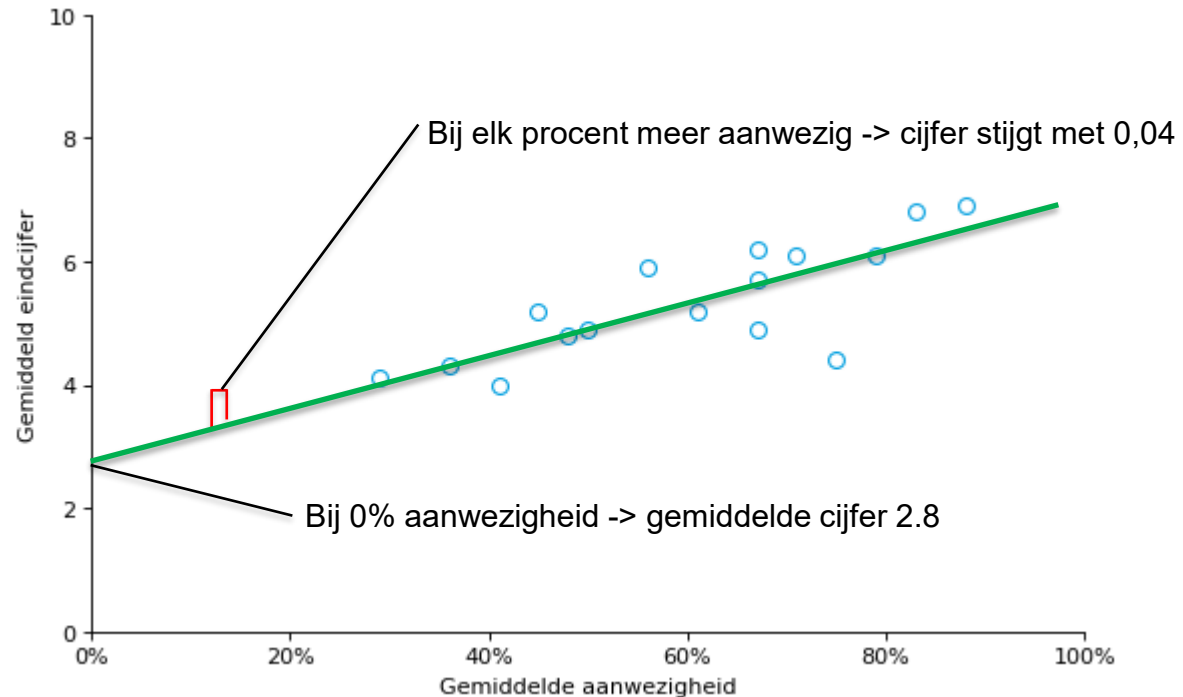
- a is het nulpunt van de grafiek
- b is de helling van de grafiek



Kleinste kwadratenmethode

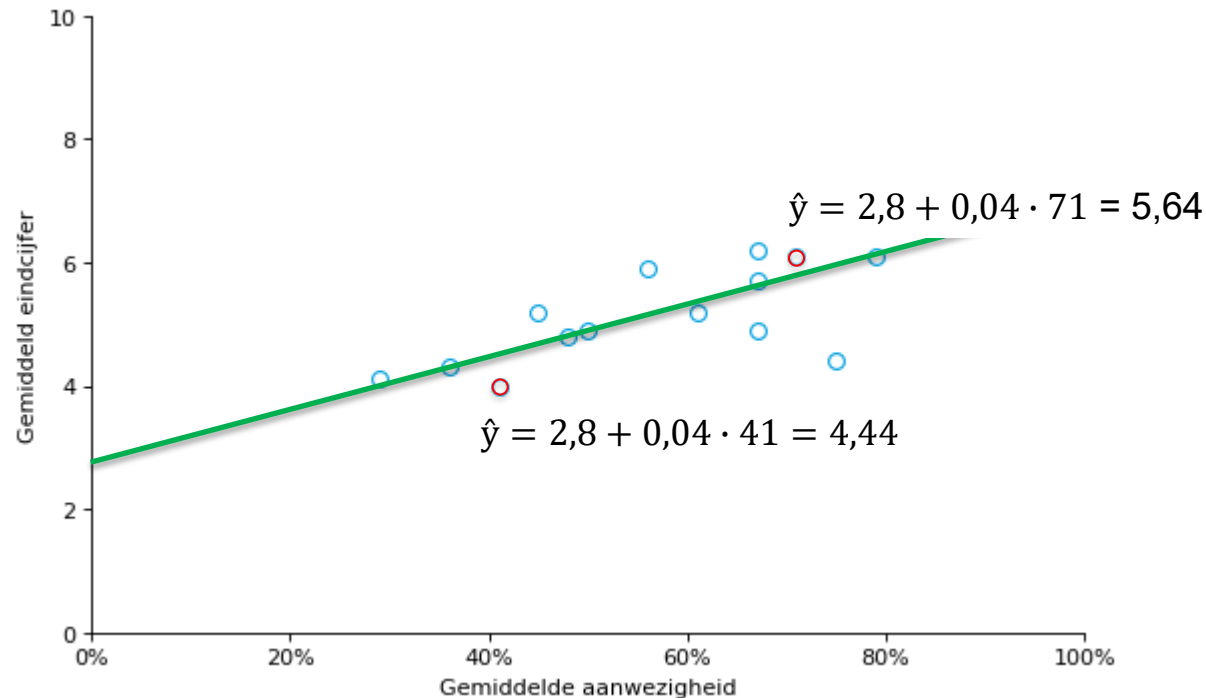
Als we onderstaande lijn invullen krijgen we

$$\hat{y} = 2,8 + 0,04 \cdot x$$



Kleinste kwadratenmethode

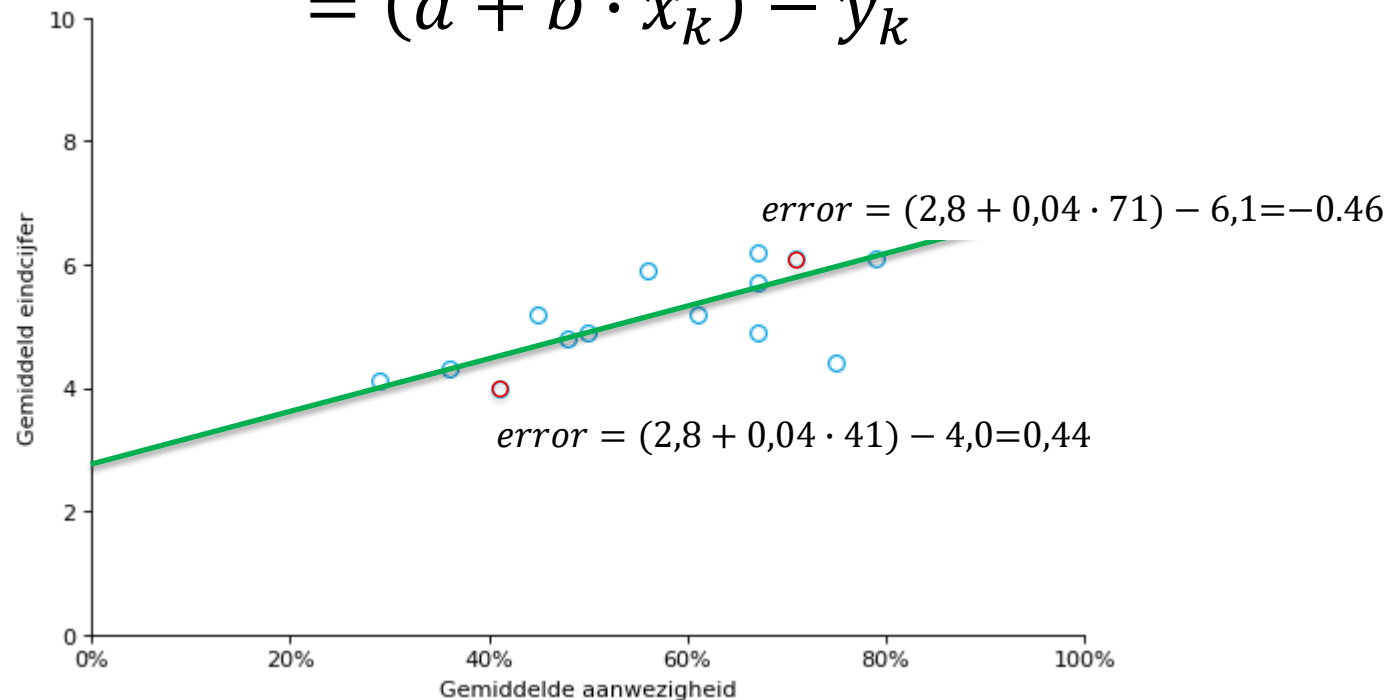
Nu kunnen we onze observaties in gaan vullen in deze formule.



Lineaire regressie

Onze formule zal waarschijnlijk niet precies de goede resultaten geven.
Deze afwijking - of error - per punt bereken je nu als volgt

$$\begin{aligned} \text{error} &= \hat{y}_k - y_k \\ &= (a + b \cdot x_k) - y_k \end{aligned}$$



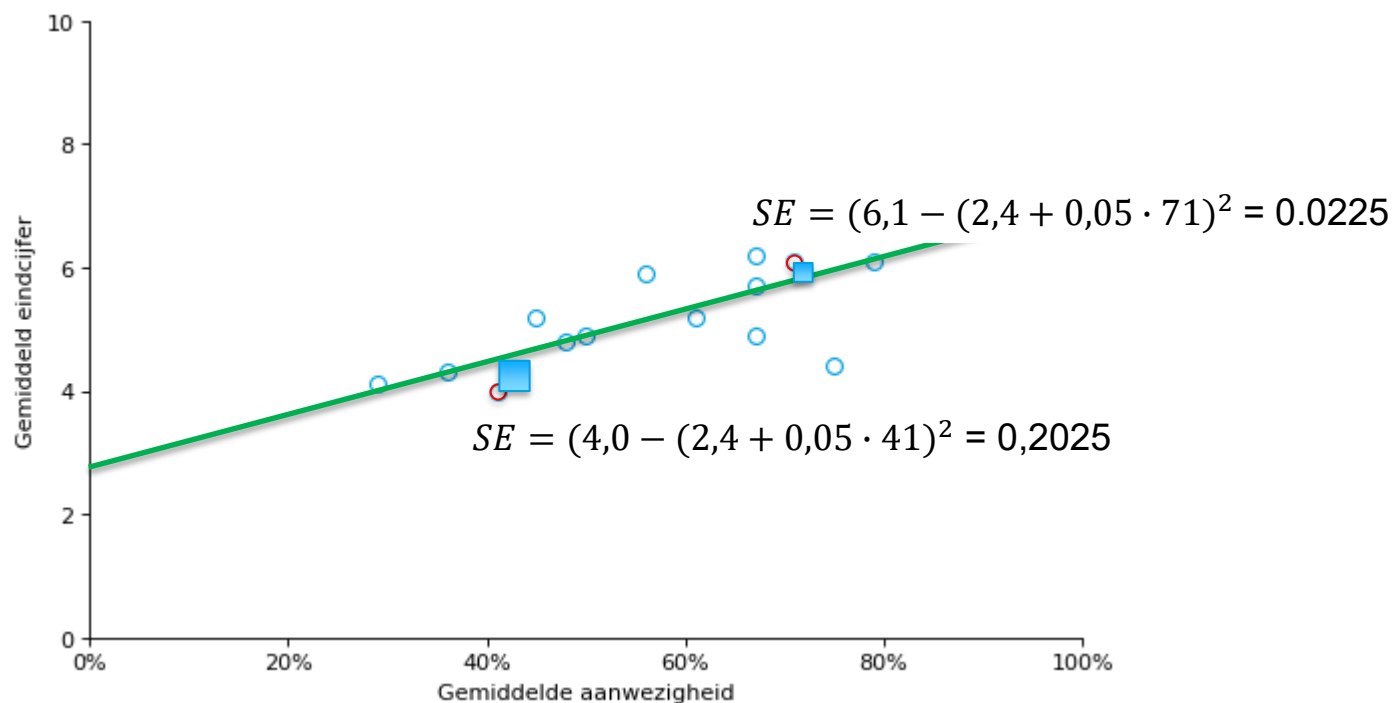
Kleinste kwadratenmethode

De vergelijking die we zoeken, **minimaliseert** de *som van kwadraten van de afwijkingen* (engels: **sum of squared errors**).

$$\begin{aligned} SSE = \sum_{k=1}^n ((a + b \cdot x_k) - y_k)^2 \\ ((a + b \cdot x_1) - y_1)^2 + \\ ((a + b \cdot x_2) - y_2)^2 + \\ ((a + b \cdot x_3) - y_3)^2 + \\ \dots \\ ((a + b \cdot x_n) - y_n)^2 \end{aligned}$$

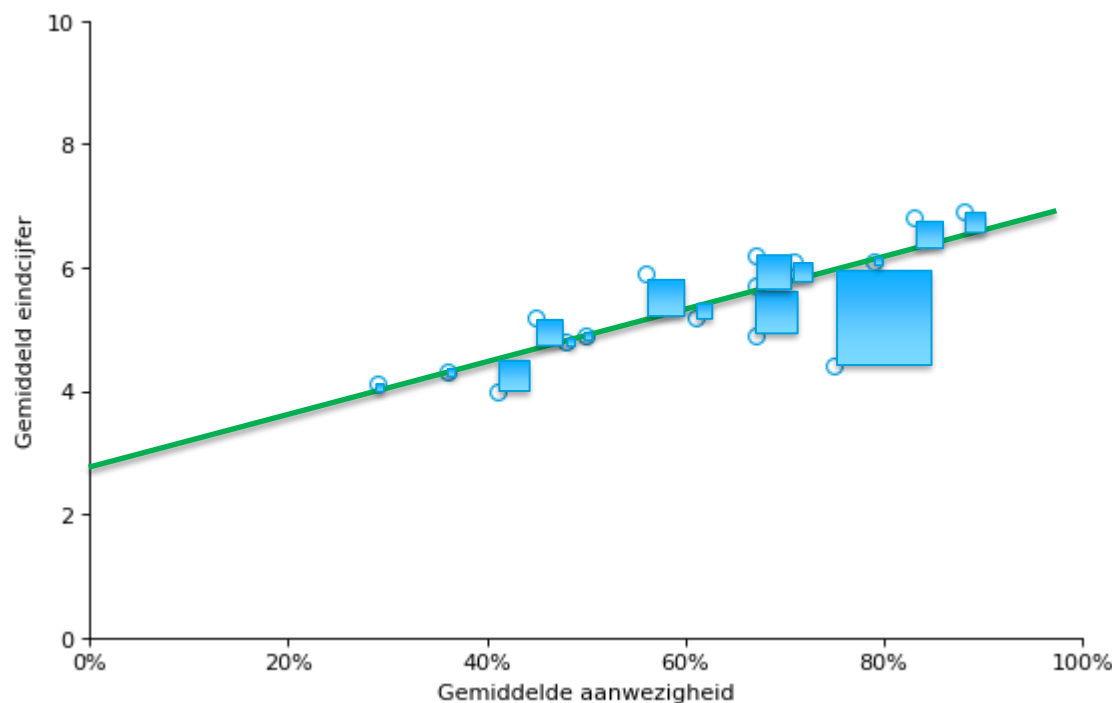
Kleinste kwadratenmethode

Door de afwijking te **kwadrateren**, houd je alleen nog **positieve** waarden over.



Kleinste kwadratenmethode

De sum of squared errors is in feite het **totale oppervlakte** van al deze vierkanten!



Oefening

Gegeven de volgende data

x	y
1	4
2	7
3	8
4	15

En de sum of squared errors

$$SSE = \sum_{k=1}^n ((a + b \cdot x_k) - y_k)^2$$

Welke SSE's krijgen we uit de volgende vergelijkingen?

- $\hat{y}_1 = 3 + 4 \cdot x$
- $\hat{y}_2 = 4 + 3 \cdot x$

Welke vergelijking voorspelt dus beter?

Error minimalisatie

Van twee lijnen kunnen we nu bepalen wat de **betere** is, maar hoe vinden we nu de **beste** lijn van *alle mogelijke lijnen*?

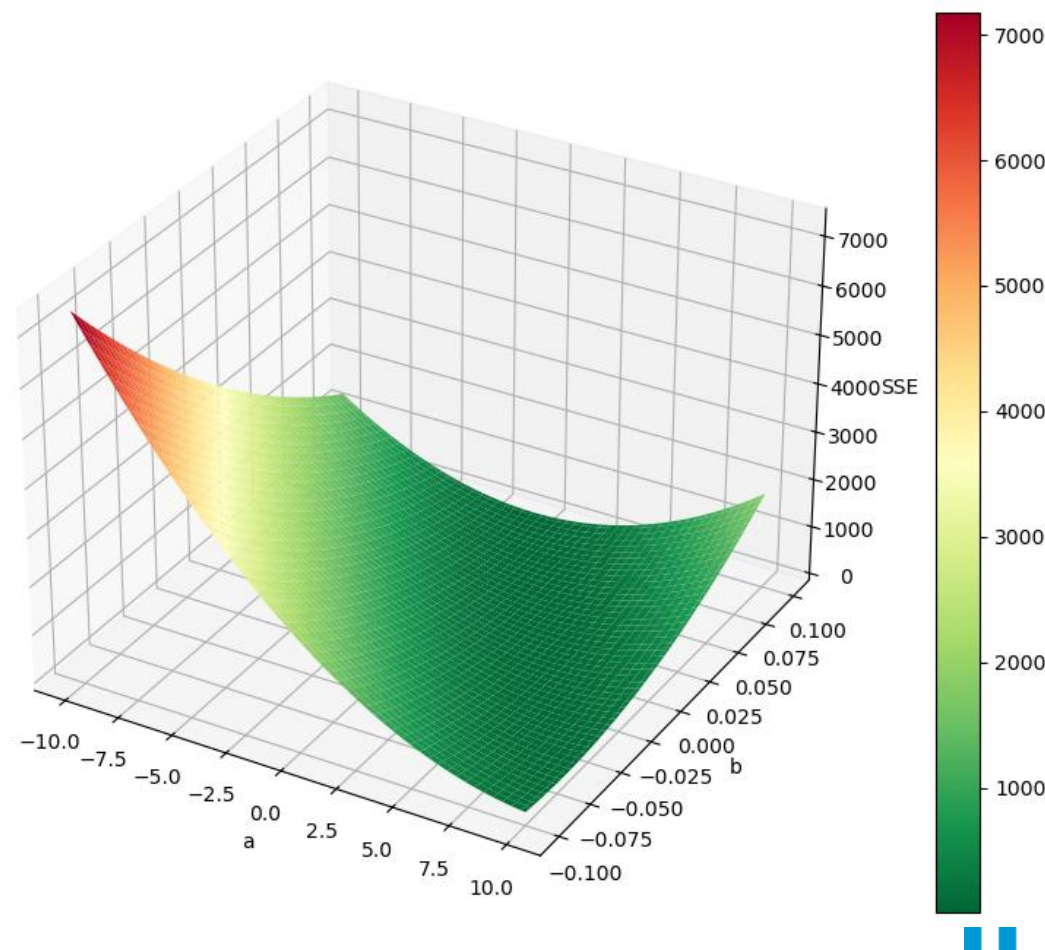
De om de beste lijn te vinden, moeten we de SSE – het totale oppervlakte van de vierkanten - **minimaliseren**.

Wat is daar voor nodig?

Error minimalisatie

Als we a , b en de SSE plotten krijgen we het volgende:

- a staat op de x-as
- b op de z-as
- De y-as geeft de SSE aan
 - Hoe lager hoe beter



Error minimalisatie

Om dit wat leesbaarder weer te geven versimpelen we

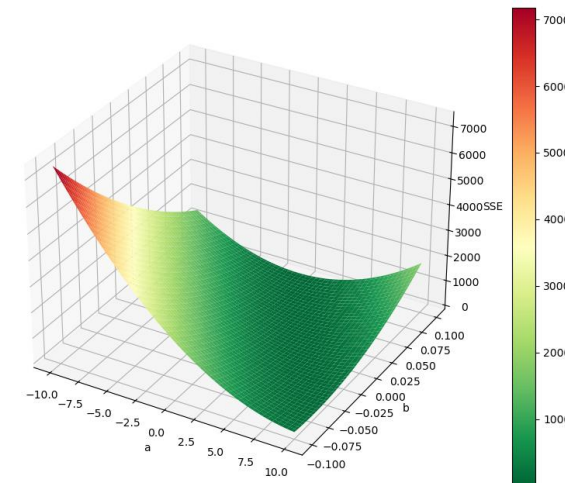
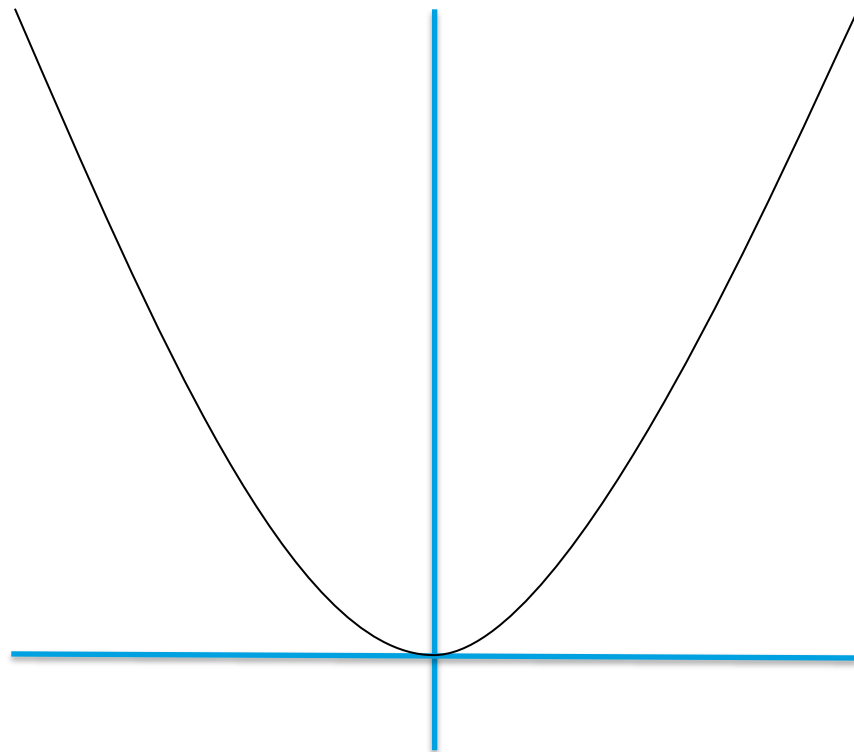
$$SSE = \sum_{k=1}^n ((a + b \cdot x_k) - y_k)^2$$

Voor het gemak even tot

$$y = x^2$$

Error minimalisatie

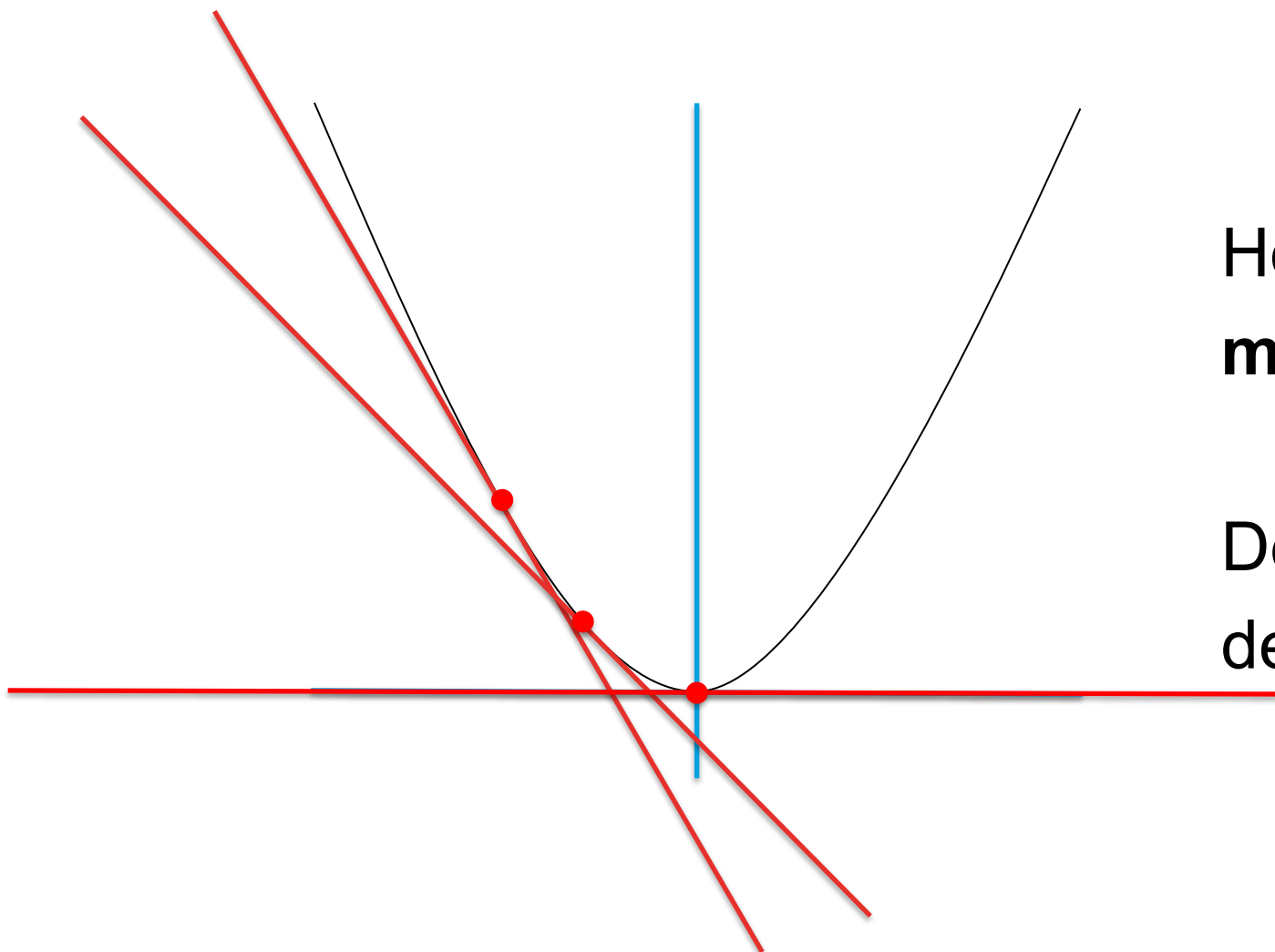
Als we $y = x^2$ plotten krijgen we



Hoe vind je het **minimum** van deze functie? Met de **helling**.

Error minimalisatie

De **helling** op een bepaald punt geeft de **maat van verandering**.

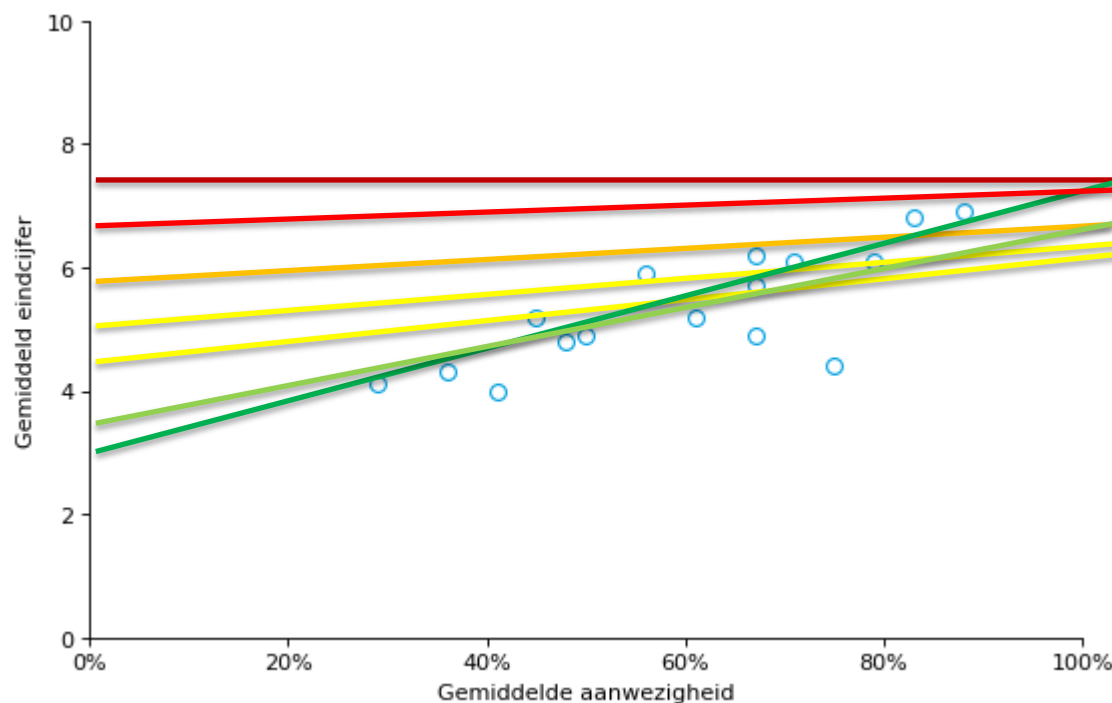


Hoe *dichter* bij het **minimum..**

Des te **horizontaler** de *helling*.

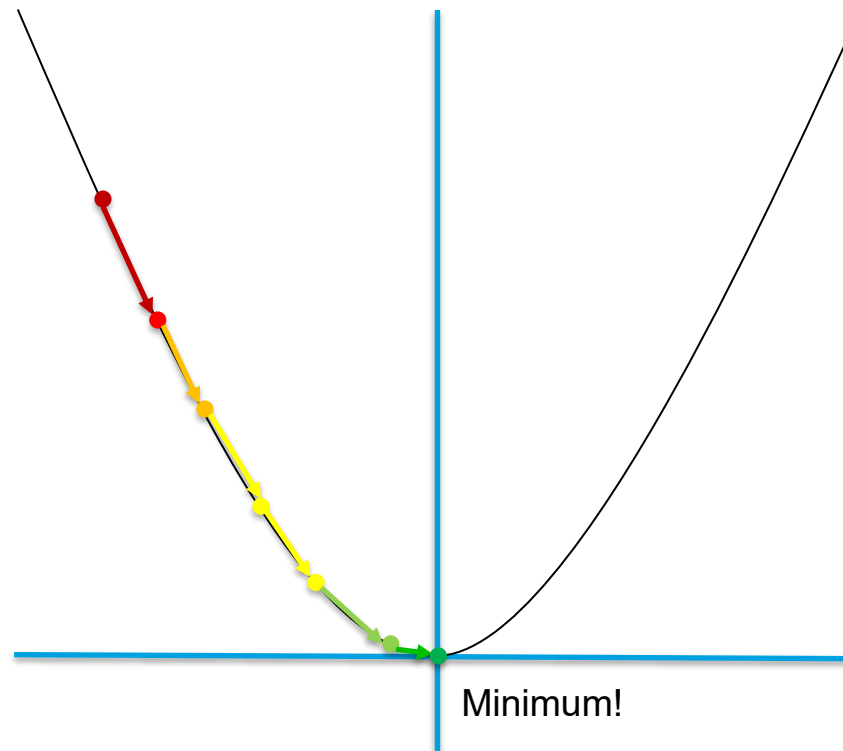
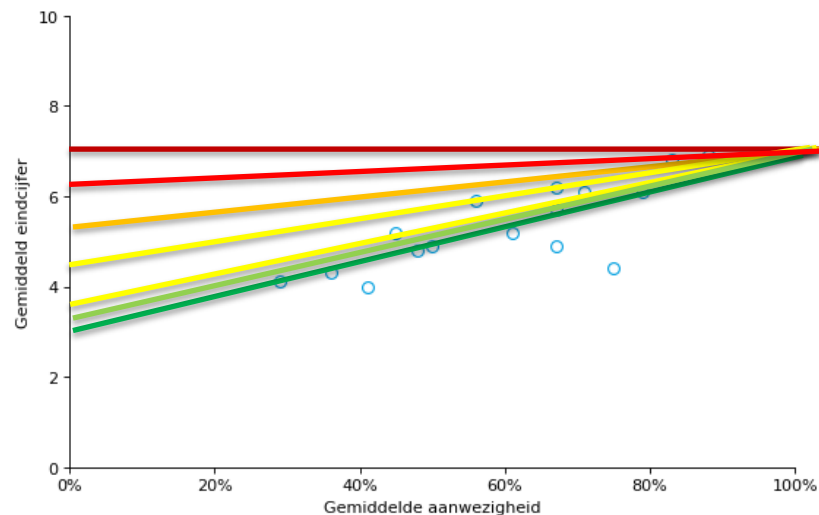
Gradient descent

Een algoritme dat hier gebruik van maakt heet **gradient descent**. Dit algoritme **gokt** een antwoord, en **verbetert** dit totdat aan bepaalde condities is voldaan.



Gradient descent

Als we de SSE van elke gok zouden *plotten*, zien we het volgende gebeuren:

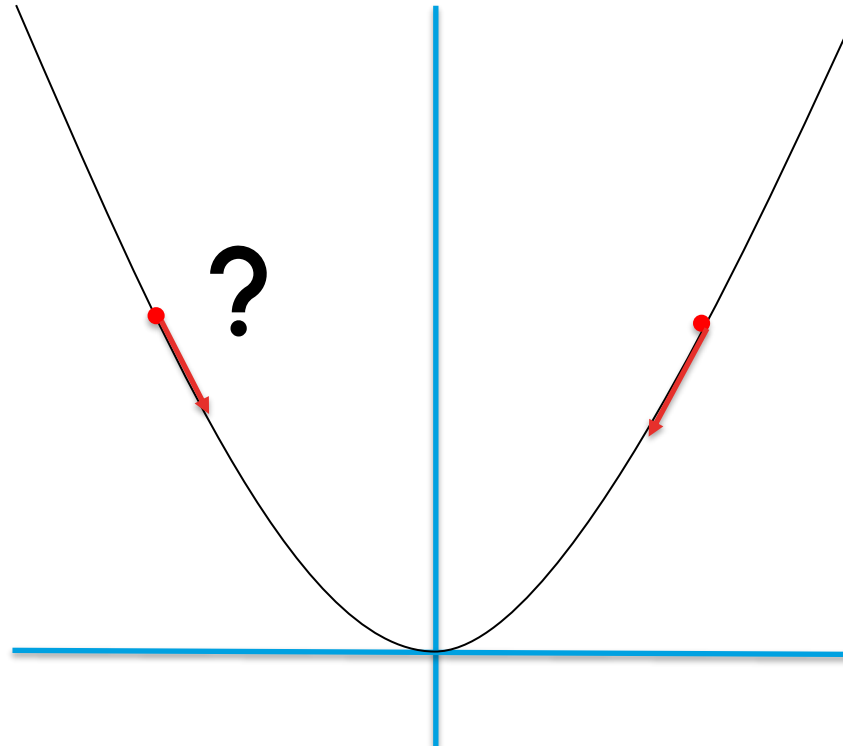


Gradient descent

Maar hoe weet het algoritme nou *welke kant* op gecorrigeerd moet worden?

- Hiervoor gebruikt het algoritme dus de **helling** van de SSE.

Is de helling
negatief? Naar
rechts!



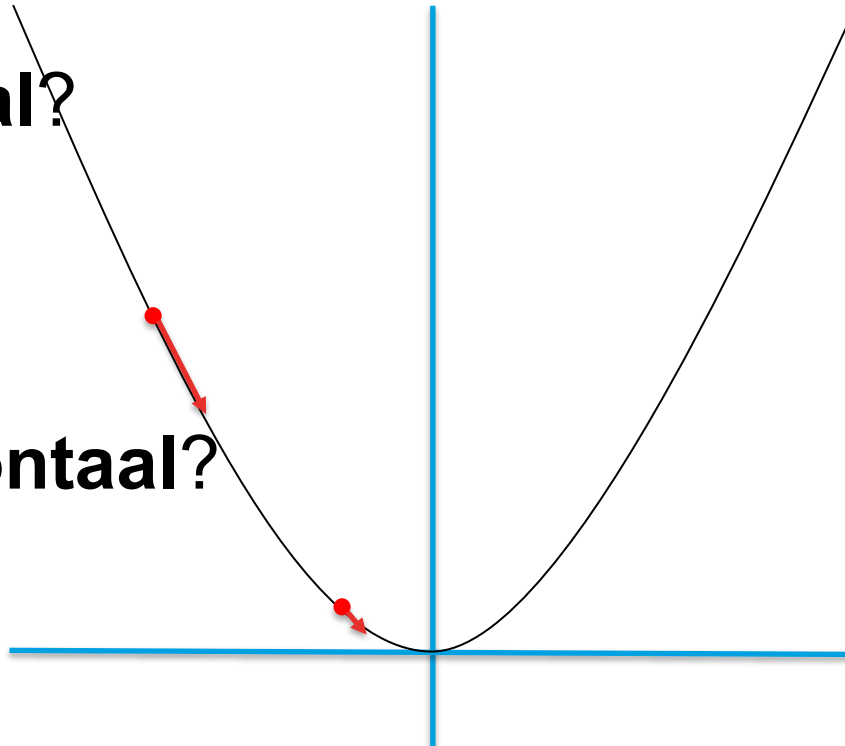
Is deze **positief**?
Naar **links**!

Gradient descent

De helling wordt ook gebruikt om te bepalen **hoe ver** het algoritme moet corrigeren

Is de helling erg **verticaal**?
Grote correctie!

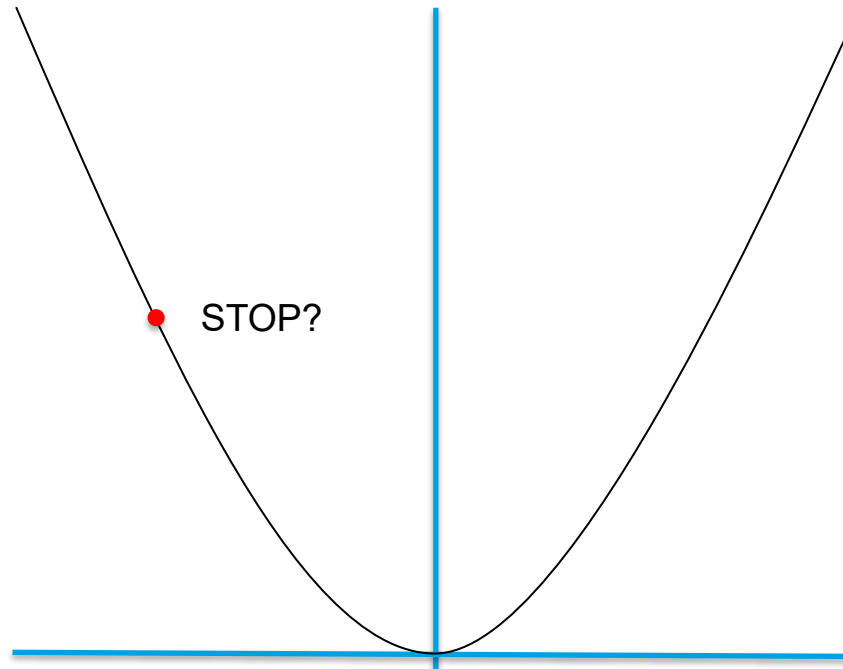
Is de helling bijna **horizontaal**?
kleine correctie!



Gradient descent

Het is niet 100% zeker dat gradient descent het minimum exact vindt...

- Daarom heeft het algoritme een **stopconditie** nodig.

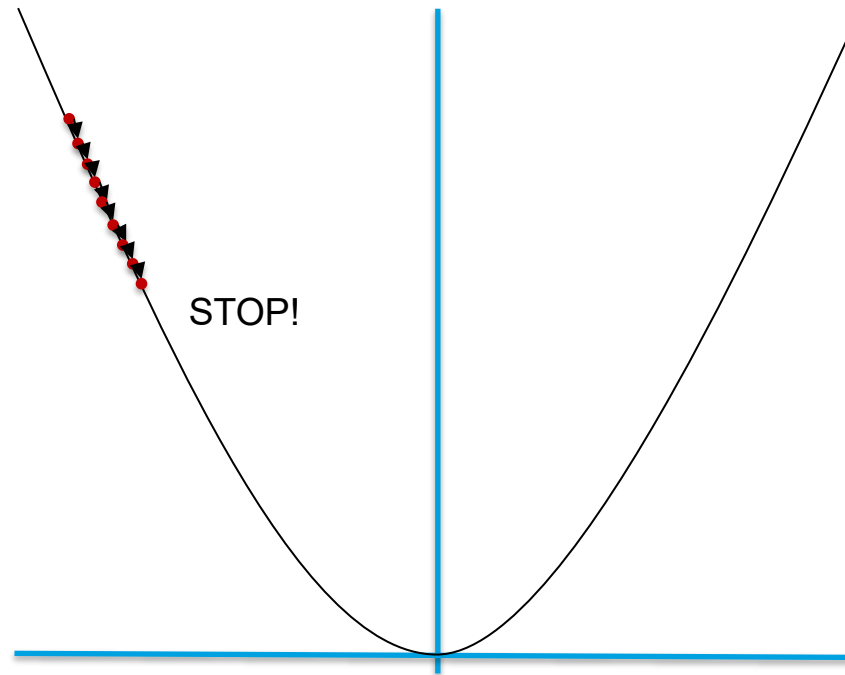
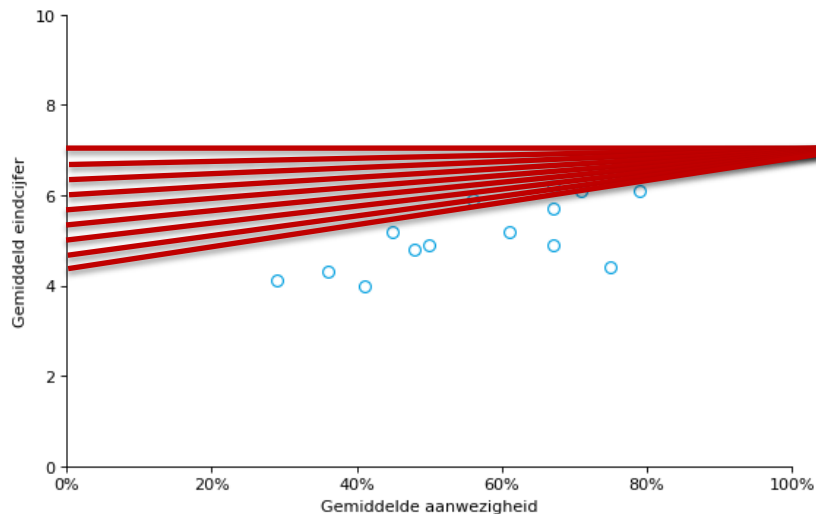


- Dit is bijvoorbeeld na *een vast aantal iteraties* zoals 500000.
- Of wanneer het model nauwelijks meer corrigeert d.m.v. een threshold.

Gradient descent

Ook krijgt gradient descent een **stapgrootte** mee.

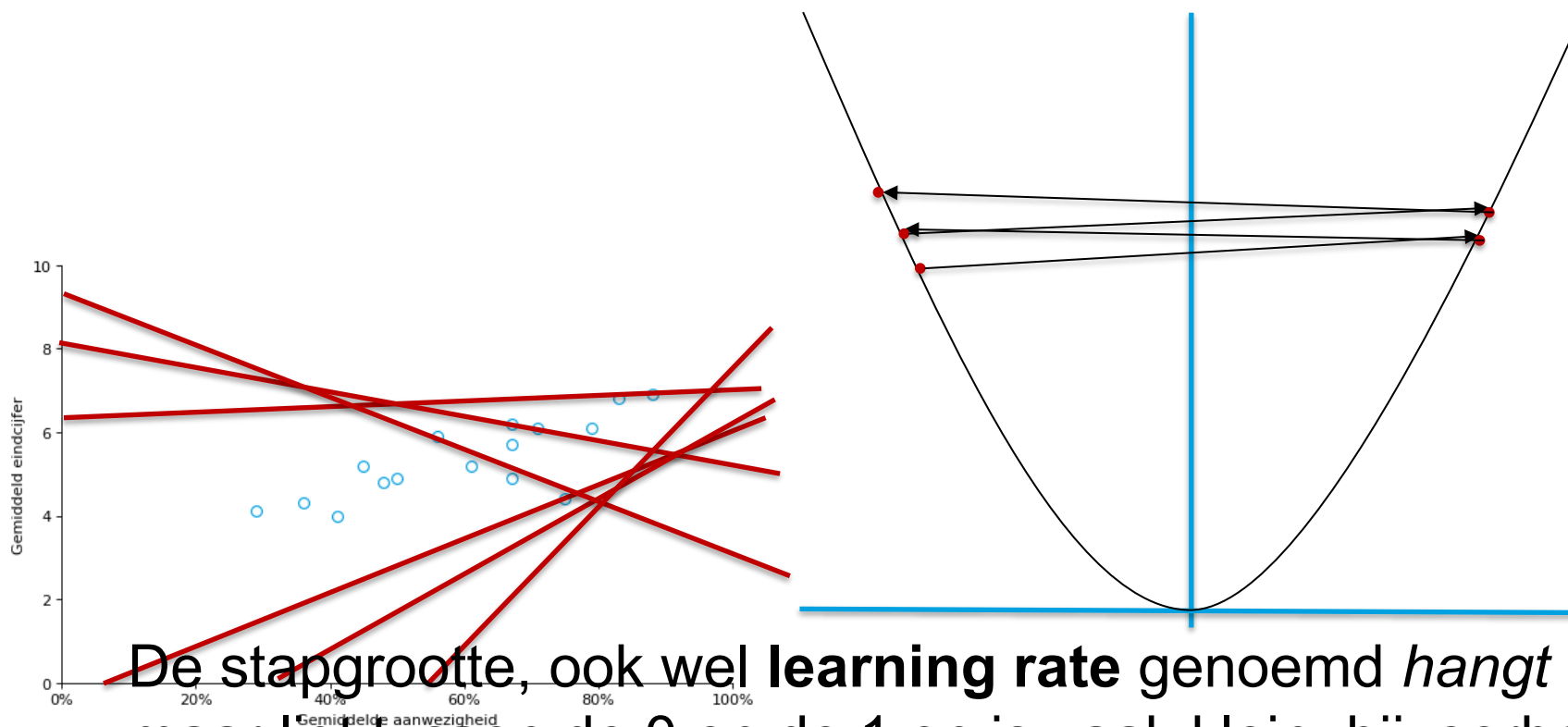
- Is deze te *klein*, dan *stopt* het algoritme *te snel*.



Gradient descent

Ook krijgt gradient descent een **stapgrootte** mee.

- Is deze te *groot*, dan *overcorrigeert* het algoritme zichzelf.

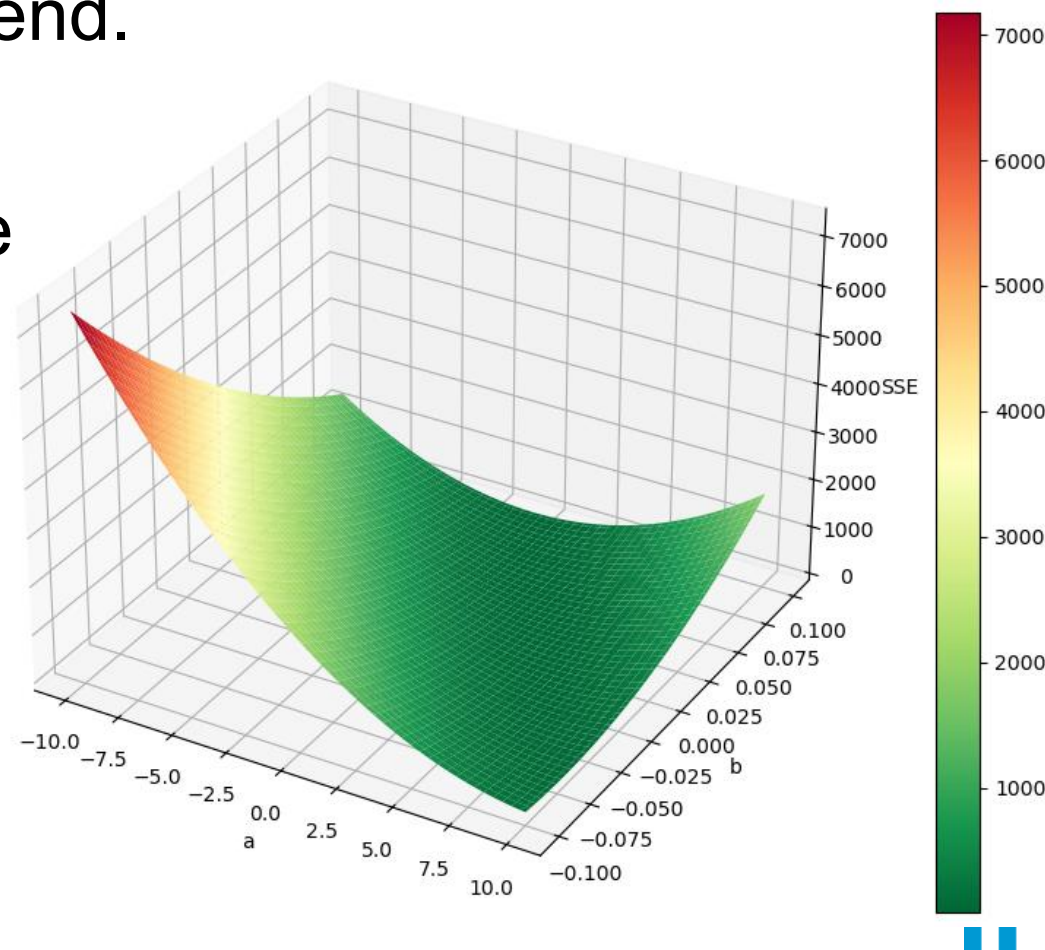


De stapgrootte, ook wel **learning rate** genoemd *hangt sterk af* van je data, maar ligt tussen de 0 en de 1 en is vaak klein, bijvoorbeeld 0.00001.

Gradient descent

Omdat we in werkelijkheid meer dan twee dimensies hebben, wordt de helling *per variabele* a en b berekend.

De combinatie van deze gedeeltelijke hellingen noemen we ook wel de **gradient**.



Gradient descent

Het gradient descent algoritme moet de volgende **stappen** zetten:

- Het algoritme moet een eerste **gok** van a en b maken.
- Het algoritme moet een gekozen aantal keer **itereren**.
 - De gradient voor a berekenen.
 - De gradient voor b berekenen.
 - De gemaakte gok voor a **aanpassen**.
 - De gemaakte gok voor b **aanpassen**.

Aan het einde benaderen a en b de **optimale** waarden. Het gradient descent algoritme **traint** dus het model $\hat{y} = a + b \cdot x$.

Pseudocode

Onderstaande **pseudocode** beschrijft het algoritme.

Initialiseer num_iterations op (bijvoorbeeld) 500000

Initialiseer learning_rate op (bijvoorbeeld) 0,00001

Initialiseer a en b op (bijvoorbeeld) 0

Itereer num_iterations keer

 initialiseer grad_a op 0

 initialiseer grad_b op 0

 Voor elke observatie (x_k, y_k)

$$\text{error} = (a + b \cdot x_k) - y_k$$

$$\text{grad}_a = \text{grad}_a + 2 \cdot \text{error}$$

$$\text{grad}_b = \text{grad}_b + 2 \cdot \text{error} \cdot x_k$$

$$a = a - \text{grad}_a \cdot \text{learning_rate} / n$$

$$b = b - \text{grad}_b \cdot \text{learning_rate} / n$$

Pseudocode

Laten we de code eens doorlopen voor 1 observatie:

Initialiseer num_iterations op (bijvoorbeeld) 500000

Initialiseer learning_rate op (bijvoorbeeld) 0,00001

Initialiseer a en b op (bijvoorbeeld) 0

Itereer num_iterations keer

 initialiseer grad_a op 0

 initialiseer grad_b op 0

 Voor elke observatie (x_k, y_k)

$$\text{error} = (a + b \cdot x_k) - y_k$$

$$\text{grad}_a = \text{grad}_a + 2 \cdot \text{error}$$

$$\text{grad}_b = \text{grad}_b + 2 \cdot \text{error} \cdot x_k$$

$$a = a - \text{grad}_a \cdot \text{learning_rate} / 1$$

$$b = b - \text{grad}_b \cdot \text{learning_rate} / 1$$

num_iterations = 500000

learning_rate = 0,00001

$a = 0, b = 0$

iteratie 0

$\text{grad}_a = 0$

$\text{grad}_b = 0$

eerste observatie = (29, 4,1)

$$\text{error} = (0 + 0 \cdot 29) - 4,1 = -4,1$$

$$\text{grad}_a = 0 + 2 \cdot -4,1 = -8,2$$

$$\text{grad}_b = 0 + 2 \cdot -4,1 \cdot 29 = -237,8$$

$$a = 0 + 8,2 \cdot 0,00001 / 1 = 0,000082$$

$$b = 0 + 237,8 \cdot 0,00001 / 1 = 0,002378$$

Oefening

Doe dit nu eens voor 2 observaties

Initialiseer num_iterations op (bijvoorbeeld) 500000

Initialiseer learning_rate op (bijvoorbeeld) 0,0001

Initialiseer a en b op (bijvoorbeeld) 0

Itereer num_iterations keer

 initialiseer grad_a op 0

 initialiseer grad_b op 0

 Voor elke observatie (x_k, y_k)

$$\text{error} = (a + b \cdot x_k) - y_k$$

$$\text{grad}_a = \text{grad}_a + 2 \cdot \text{error}$$

$$\text{grad}_b = \text{grad}_b + 2 \cdot \text{error} \cdot x_k$$

$$a = a - \text{grad}_a \cdot \text{learning_rate} / 2$$

$$b = b - \text{grad}_b \cdot \text{learning_rate} / 2$$

num_iterations = 500000

learning_rate = 0,0001

$a = 0, b = 0$

iteratie 0

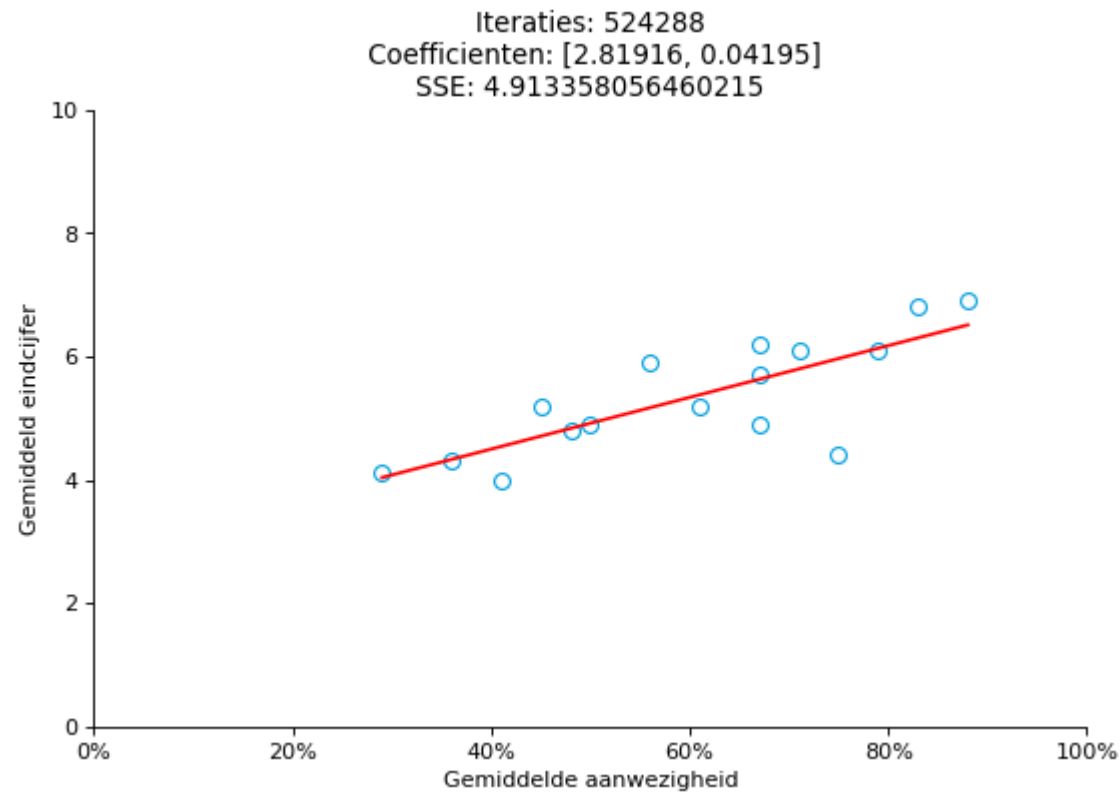
$\text{grad}_a = -8,2$

$\text{grad}_b = -237,8$

tweede observatie = (36, 4,3)

Resultaten

Na zo'n 500.000 iteraties zien we het volgende:



Het lineaire regressiemodel *convergeert* op **a = 2,8** en **b=0,04**

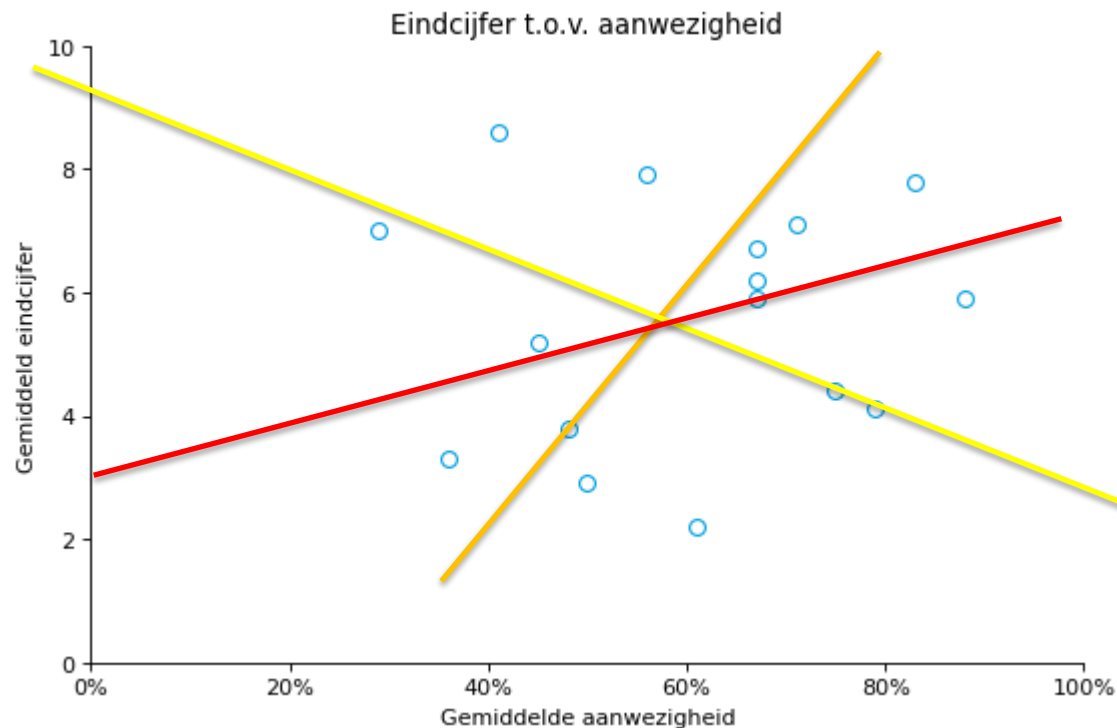
Oefening

- Implementeer de functie `gradient_descent` op basis van de gegeven pseudocode
 - Oefening AI6.1: Lineaire regressie en gradient descent (zie Canvas)

Correlatie

Stel nou dat de datapunten er als volgt uit hadden gezien.

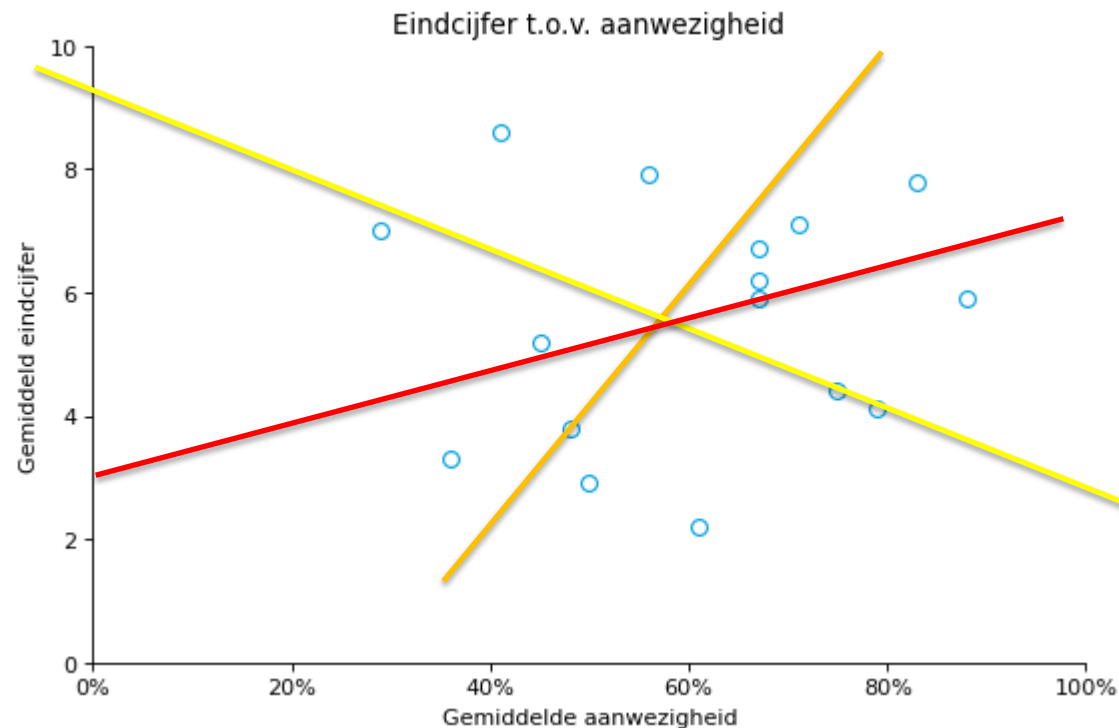
Hoe zou je *intuïtief* de regressielijn trekken?



Alle lijnen lijken even
nietszeggend...

Correlatie

We zeggen: het lineaire *verband* tussen de twee variabelen is laag.
Maar hoe bepalen we dit? Met de **correlatie**!



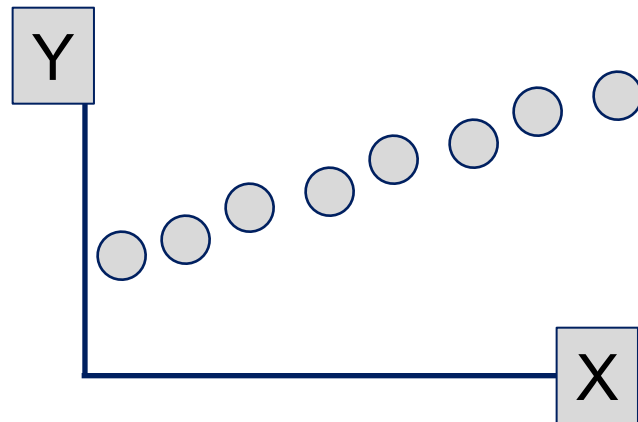
Correlatie

De correlatie meet de *mate van samenhang* tussen twee variabelen.

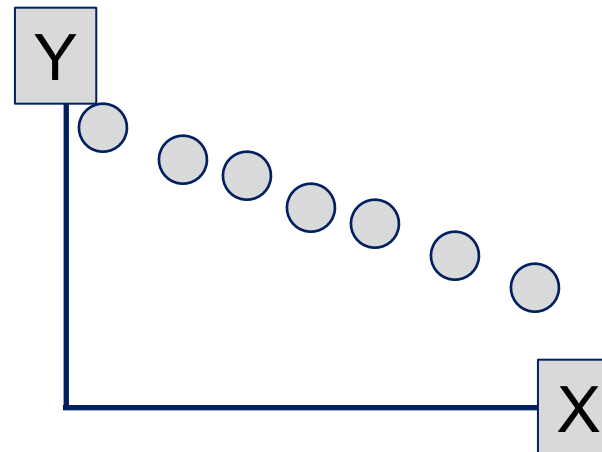
De lineaire correlatie wordt uitgedrukt in de **correlatiecoëfficiënt r** .

- De r ligt altijd tussen -1 en $(+)1$.
- Het plus- of minteken geeft de richting van de correlatie aan.

Positieve
correlatie
($r > 0$)



Negatieve
Correlatie
($r < 0$)

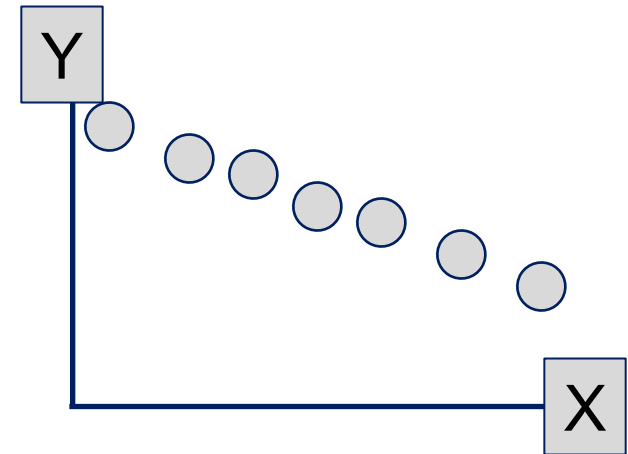


Correlatie

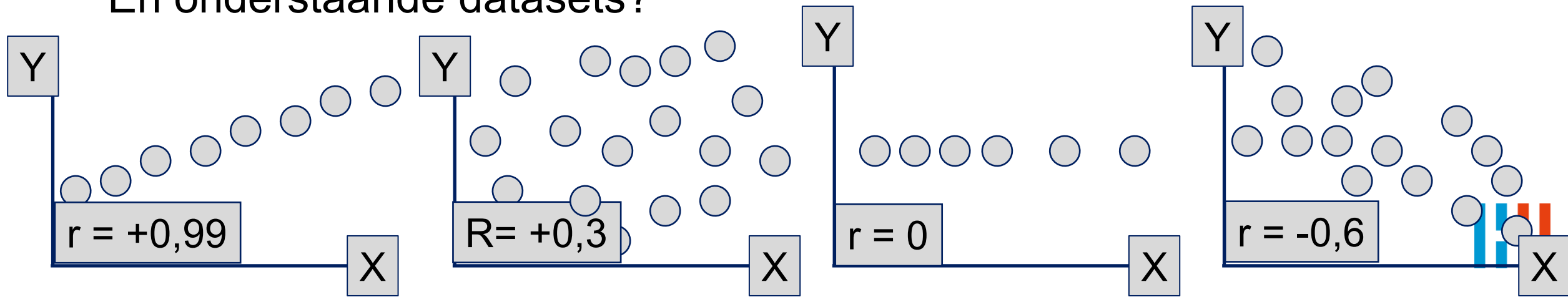
De *absolute waarde* van correlatiecoëfficiënt r geeft de **sterkte** van de correlatie weer: hoe groter $|r|$, des te sterker de samenhang!

Deze dataset heeft een r van zo'n -0,99:

- Oftewel: **sterk** negatief gecorreleerd.



En onderstaande datasets?



Correlatie

Wanneer correleren twee variabelen nou voldoende?

- Er is geen absolute waarheid, dit verschilt *per vakgebied*.
- Een veelgebruikte verdeling is:
 - Perfecte correlatie: $|r| = 1$
 - Sterke correlatie: $1 > |r| \geq 0,5$
 - Matige correlatie: $0,5 > |r| \geq 0,3$
 - Zwakke correlatie: $0,3 > |r| > 0$
 - Geen correlatie: $|r| = 0$

Pearsons correlatiecoëfficiënt

De meest gebruikelijke methode om lineaire correlatie te berekenen is met de **Pearsons correlatiecoëfficiënt**.

De formule hiervan ziet er nogal indrukwekkend uit:

$$\frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2}}$$

Toch kennen we alle wiskunde wel.

- $\bar{x}$ is het gemiddelde van x , en $\bar{y}$ is het van y (eerder: μ).
- $\sum_{i=1}^n$ sommeert voor alle i van 1 tot en met n .

Pearsons correlatiecoëfficiënt

Sterker nog, de *noemer* bestaat uit twee bekende formules!

$$\frac{\dots}{\sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2}}$$

Welke twee (dezelfde) formules herkennen we hier?

- De noemer is 2x de **standaarddeviatie**: $\sqrt{\frac{\sum_{i=1}^n (x_i - \mu)^2}{n}}$

Door te delen door deze standaarddeviaties eindigen we altijd met een waarde tussen de -1 en de 1!

Pearsons correlatiecoëfficiënt

Nu blijft alleen nog de *teller* over, die is ook niet zo spannend:

$$\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

...

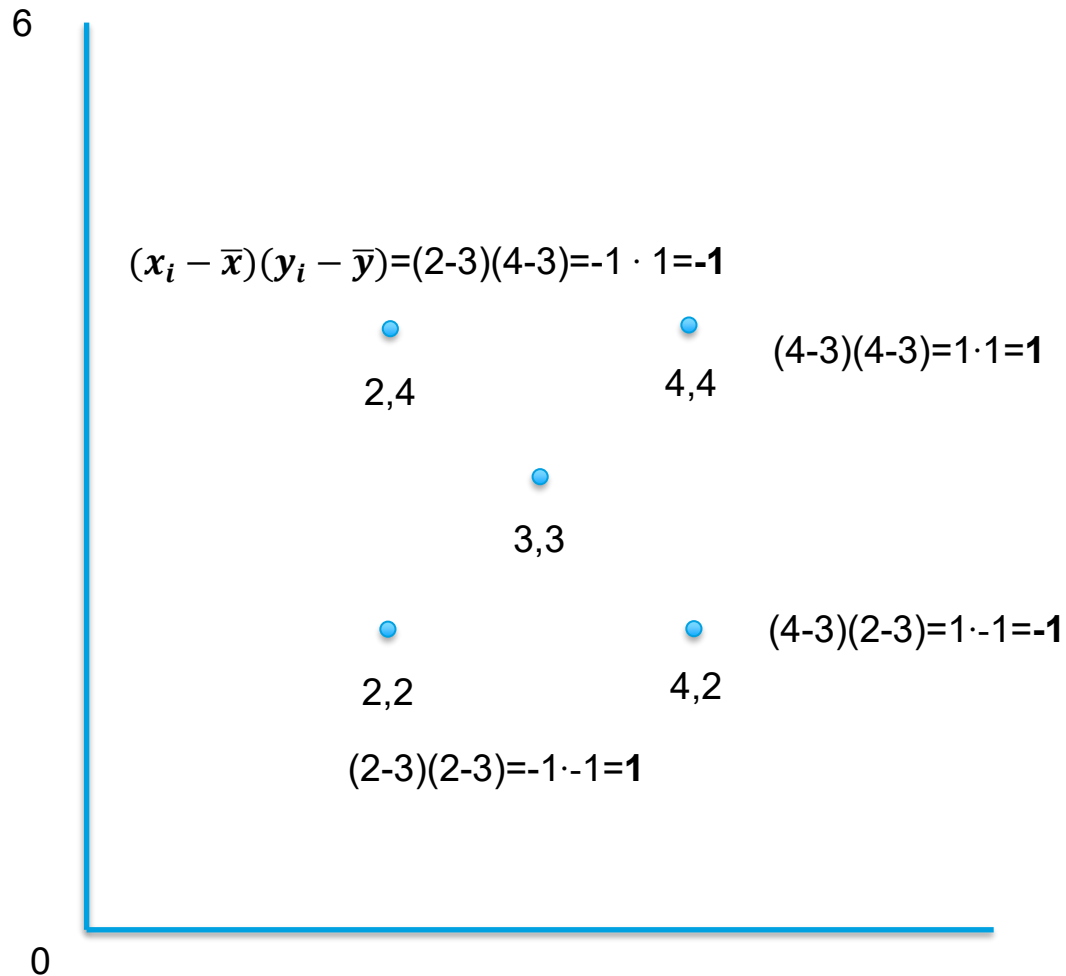
Wat gebeurt er in de teller?

- Hier wordt berekend in welke mate variabelen *met elkaar veranderen*.
 - Twee positieve afwijkingen t.o.v. μ worden samen *positief*.
 - Twee negatieve afwijkingen t.o.v. μ worden samen *positief*.
 - Een negatieve en een positieve afwijking wordt samen *negatief*.

Dit wordt ook wel de **covariantie** genoemd.

De Pearsons correlatiecoëfficiënt is dus de *genormaliseerde covariantie*!

Pearsons correlatiecoëfficiënt



$$\frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2}}$$

Aangezien onder de streep alleen normalisatie is, zit de truc hem in de Vergelijking boven de streep.

Stel, het gemiddelde is (3,3)

- (2,4) leidt dan tot een negatief getal, en de lijn tov (3, 3) daalt ook
- (4,4) leidt dan tot een positief getal, en de lijn tov (3, 3) stijgt ook
- (2,2) leidt dan tot een positief getal, en de lijn tov (3, 3) stijgt ook
- (4,2) leidt dan tot een negatief getal, en de lijn tov (3, 3) daalt ook

Pearsons correlatiecoëfficiënt

Voordat we met de Pearsons correlatiecoëfficiënt gaan werken, kunnen we de formule nog wat versimpelen.

$$\frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2}}$$

Is namelijk gelijk aan

$$\frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}$$

Dat scheelt straks een hoop berekeningen!

Oefening

$$\frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}$$

Bereken de Pearsons correlatiecoëfficiënt voor ons proefdatasetje

x	y
1	4
2	7
3	8
4	15

Huiswerk

- Maak `gradient_descent()` af
 - Oefening AI6.1: Lineaire regressie en gradient descent (zie Canvas)
- Werk aan Opdracht AI2: Data science (zie Canvas)